FluxShard: Motion-Aware Feature Cache Reuse for Edge-Cloud Video Analytics

Anonymous

***Abstract*—Caching and reusing intermediate features across consecutive frames is a common technique for reducing redundant computation and transmission in edge-cloud video analytics. Existing methods manage the cache in a fixed, whole-scene coordinate system and assume a largely stationary scene. Under real-world motion, a tight reuse threshold discards large portions of the cache even when most content has merely shifted spatially, while a loose threshold allows stale features to corrupt the output. The root cause is that the cache is managed as an indivisible unit, yet real-world motion varies across the scene.**

We present FluxShard, a motion-aware edge-cloud analytics system that uses codec-level block motion vectors (MVs) as a control signal for feature cache reuse. FluxShard realigns cached features at the block level along their motion vectors to separate displaced content from genuinely changed regions, producing a minimal residual set. The Receptive Field Alignment Principle (RFAP) guarantees reuse correctness under heterogeneous motion with negligible overhead. An MV-guided cache remapping scheme keeps the feature cache aligned with each new frame, sustaining efficient reuse across consecutive frames. At runtime, FluxShard routes the sparse residual workload between edge and cloud based on network conditions and device load. Evaluation on instance segmentation and pose estimation with YOLO11 on two dynamic video benchmarks across various network conditions shows that FluxShard achieves 32.6–83.8% latency reduction and 14.9–64.0% energy savings over all baselines while retaining 97.3–98.0% of dense-model accuracy.

***Index Terms*—Video analytics, robotics, edge-cloud collaborative system, CNN inference acceleration**

I. Introduction

Video analytics uses deep neural networks (DNNs) to process continuous video frames and produce results such as object detections, segmentation masks, and pose estimates. Video analytics is a core building block of intelligent edge applications: autonomous drones [1], [21], embodied robots [27], [10], augmented-reality headsets [28], and smart surveillance cameras [13] all rely on video analytics to perceive and react to their surroundings in real time. Running these models directly on the edge device avoids uploading raw video and keeps latency predictable, but edge hardware offers limited compute and energy. A common alternative is to offload frames to a powerful cloud server; however, streaming high-resolution video over a bandwidth-constrained wireless uplink introduces transmission delay that can easily exceed the real-time budget. Neither naive local nor remote processing reliably meets the joint latency and accuracy demands of these applications.

A natural way to reduce both transmission and computation is to exploit temporal redundancy: consecutive frames share most of their content, and this redundancy extends to intermediate feature maps. Reuse-based methods cache and reuse previous computed feature maps and recompute only the changed portions, reducing both transmitted data and computation simultaneously.

However, existing reuse methods are designed for fixed cameras and struggle with the complex motion patterns introduced by mobile edge devices. Some methods reuse whole intermediate feature maps only when consecutive frames exceed a global similarity threshold [17], [8]. Others compute per-pixel differences and recompute only positions where the difference exceeds a threshold [2], [19], [24], [23]. Despite their different granularities, these methods share a common limitation: they treat the cached feature maps as an indivisible whole; the non-uniform motion patterns in real-world scenes invalidate large portions of the cache even when most content has merely shifted spatially, wasting computation and bandwidth. A cache reuse mechanism that adapts to local, heterogeneous motion is therefore needed for mobile video analytics.

Our key insight is that block-level motion vectors (MVs) from standard video codecs provide exactly such a per-region motion signal at zero additional cost. MVs are block-level displacements that map each current-frame block (e.g., 16×16 pixels) to the reference frame. Because MVs capture geometric displacement rather than content change, re-indexing cached feature-map blocks along their MV trajectories recovers reusable content that whole-scene methods would discard. The remaining positions where MVs cannot account for the actual change (e.g., high difference in pixel values after alignment due to non-rigid deformation, dis-occlusion, or newly appearing content) form the *recomputation set*: the minimal set of positions that must be freshly computed or transmitted.

Realizing this idea introduces two challenges. First, *receptive field inconsistency*: even when every pixel individually matches its MV-aligned counterpart, the reused output can still be wrong. Operators like 3×3 convolutions compute each output from a spatial neighborhood (the receptive field); correctly reusing a cached output requires verifying that the entire neighborhood matches, and this check at every layer and position costs as much as recomputation, negating the latency benefit of reuse. Second, *cache maintenance*: existing methods use in-place cache updates assuming a static coordinate system; under per-block motion the cache drifts out of alignment, and writing recomputed values back along MVs creates write conflicts (e.g., multiple blocks are mapped to a same cache position) and staleness (cache positions that no blocks are mapped to are not updated). These errors compound across frames, progressively eroding reuse opportunities until caching offers no benefit over dense execution.

We propose FluxShard, a motion-aware edge-cloud video analytics system that achieves low latency and high accuracy by managing feature cache reuse at the granularity of individual motion regions. To address receptive field inconsistency, FluxShard formalizes the Receptive Field Alignment Principle (RFAP), which folds the per-layer, per-position neighborhood check into a single pass over the input-level MV field, identifying positions that must be recomputed. To address cache maintenance, FluxShard adopts a remap-then-update rule analogous to video codec frame reconstruction: backward MVs warp the cache to the current coordinate system, and freshly computed values are merged in, producing a valid reference without write conflicts or stale entries. These mechanisms determine how much work remains after safe reuse; a profiling-driven dispatcher then routes that work to the cheaper endpoint.

In summary, this paper makes the following contributions:

- We identify the granularity mismatch between whole-scene cache management and local heterogeneous motion as the key limitation of existing feature reuse methods, and show that codec-level motion vectors provide a free, per-block geometric signal to overcome it.
- We propose FluxShard, a motion-aware edge-cloud video analytics system that manages feature cache reuse at the granularity of individual motion regions and routes the resulting recomputation workload to the cheaper endpoint.
- We develop the Receptive Field Alignment Principle (RFAP) to guarantee reuse correctness, motion-aware cache remapping to maintain cache validity across frames, and a profiling-driven dispatch scheduler to minimize per-frame latency.
- Evaluation on real-world dynamic video benchmarks shows that FluxShard achieves 32.6–83.8% latency reduction and 14.9–64.0% energy savings over all baselines while retaining 97.3–98.0% of dense-model accuracy.

In the rest of this paper, §II motivates the problem with empirical evidence. §III defines the system model and pipeline. §IV develops the reuse criterion, RFAP, cache remapping, and dispatch mechanisms. §V describes the implementation. §VI evaluates FluxShard against four baselines. §VII discusses related work and §VIII concludes.

II. Challenges and Motivation

A. The Edge-Cloud Dilemma

Edge-cloud video analytics faces two competing bottlenecks: local compute and network transmission. Running YOLO11m-pose [16] on an NVIDIA Jetson Xavier NX requires approximately 446.8 ms per 1024×1024 frame, far exceeding real-time requirements. Offloading every frame eliminates the compute bottleneck but introduces a transmission bottleneck: an uncompressed 1024×1024 frame is about 3 MB; at 30 fps this sustains $\sim$755 Mbps, routinely exceeding edge uplink capacity. The cloud server finishes inference in only about 27.6 ms, yet end-to-end offload latency reaches approximately 290.3 ms under a typical 5G uplink [20] ($\sim$383 Mbps). Figure 1a quantifies this tension.

Feature cache reuse can reduce both bottlenecks: when consecutive frames share content, cached results substitute for fresh transmission and computation. Existing mechanisms [17], [8], [24], [23] treat the cache in a fixed, whole-scene coordinate system, whether the reuse decision is binary (reuse or recompute the entire frame) or pixel-level (propagate a dense difference map). This granularity mismatch breaks down under motion. Figure 1b shows that even modest displacement triggers widespread invalidation: the reuse ratio of DeltaCNN [24] and M-DeltaCNN [23] drops from over 60% on near-static sequences to below 25% once motion exceeds 20 px/frame, despite most content being merely shifted rather than truly changed.

Takeaway. Cache reuse is essential, but whole-scene granularity cannot tolerate motion. A finer-grained, motion-aware mechanism is needed.

B. Motion Vectors: Opportunity and Correctness Failure

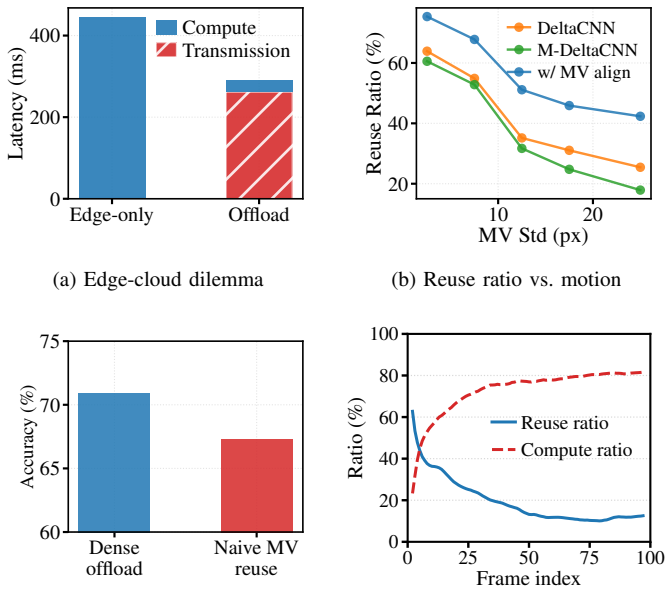
One such motion signal already exists in the video pipeline. H.264/H.265 codecs estimate a per-block displacement as part of normal encoding, providing per-region motion information at zero additional cost. Although lossy codec stages (transform, quantization) can distort feature statistics [7], [6], [4], the motion vectors themselves are a purely geometric signal unaffected by these stages. By shifting cached feature blocks according to these MVs, displaced content is realigned before differencing, and the remaining recomputation set concentrates on regions that MVs cannot explain: dis-occlusion, deformation, and newly appearing content. As shown in Figure 1b, MV alignment recovers much of the lost reuse, maintaining over 55% even under strong motion where DeltaCNN and M-DeltaCNN fall below 25%. These results establish that MV alignment can recover most displaced content; the remaining question is whether the aligned features are *correct* for downstream inference.

Despite the improved reuse ratio, directly reusing MV-aligned features still produces severe accuracy loss. Figure 1c shows that accuracy (measured by Object Keypoint Similarity, OKS) drops from 70.95% to 67.31% under naïve MV reuse, a 5.13% relative degradation. The root cause is receptive field inconsistency: layers whose receptive field spans more than one position aggregate spatial neighborhoods. When adjacent blocks carry different MVs, re-indexing assembles patches that were never contiguous in the original frame; the cached output was computed from a different neighborhood. The resulting error compounds through cascaded layers.

Takeaway. MV alignment eliminates most false cache misses, but exploiting it requires a correctness guarantee under heterogeneous per-block motion. This is the first challenge our design must address.

C. Naïve Cache Update Breaks Future Reuse

Even with a correct recomputation set, the system still faces a second problem: how to maintain the cache after sparse recomputation. Existing reuse systems simply overwrite the cache in place. That policy assumes the cache and the current frame still share the same coordinate system. Under heterogeneous per-block motion, however, the cache remains



(a) Edge-cloud dilemma (b) Reuse ratio vs. motion

(c) Accuracy under naive reuse (d) Naive cache update drift

Figure 1. Motivation for FluxShard. All panels use pose estimation (YOLO11m-pose) on 3DPW under medium bandwidth with an NVIDIA Jetson Xavier NX as the edge device and an RTX 3080 as the cloud server. (a) End-to-end latency for dense local vs. dense offload inference. (b) Reuse ratio vs. per-frame motion intensity (MV standard deviation) for DeltaCNN, M-DeltaCNN, and MV-aligned reuse. (c) Accuracy (OKS) of naïve MV reuse without correctness checking vs. dense offload. (d) Reuse ratio and compute ratio over the first 100 frames of a representative sequence without cache remapping.

anchored to the previous frame while the current content has moved to new locations.

A natural extension is to write each newly recomputed recomputed block back to the cache location indicated by its MV. This also fails: the mapping from current blocks to cached locations is many-to-one and incomplete, creating write conflicts at overlapping targets and leaving stale content at unmapped locations. Positions that should remain reusable no longer find a valid history entry, while positions declared reusable may read an incorrect value.

Once written into the history, these errors accumulate. Figure 1d tracks this on a representative 1660-frame pose estimation sequence. Without proper cache realignment, the reuse ratio drops from over 70% to below 15% and the compute ratio rises from 13% to over 80%, approaching dense execution cost. Genuinely reusable content persists but no longer aligns with the current coordinate system.

Takeaway. Correct recomputation set extraction is not enough. MV alignment can isolate a small recomputation set, but without first realigning the cache to the current frame layout, sparse updates leave behind stale or conflicting history. That broken history enlarges the next recomputation set and corrupts nominal reuse. Cache maintenance under heterogeneous motion is therefore the second challenge our design must address.

III. System Overview

This section defines the system model and design goal (§III-A), then describes the end-to-end pipeline (§III-B).

A. System Model

We consider an edge-cloud video analytics system in which an edge device (e.g., an NVIDIA Jetson) receives a live video stream $\{I_t\}_{t=1}^{T}$ from a co-located camera, and a remote cloud server provides additional compute capacity. The two endpoints communicate over a time-varying bandwidth-limited uplink. A block-level motion vector (MV) field $\mathbf{m}_t$ is extracted from the video stream for each frame I_t. For every pixel position (i, j) in I_t, $\mathbf{m}_t(i, j)$ gives the displacement to its corresponding position $(\hat{\imath}, \hat{\jmath}) = (i, j) - \mathbf{m}_t(i, j)$ in the reference frame I_{t-1}; all pixels within the same 16×16 block share one displacement. Each endpoint hosts an identical copy of the inference model and maintains a per-layer feature cache of intermediate features from its most recent inference. For each incoming frame I_t, FluxShard executes the recomputation workload at one endpoint: either locally at the edge or remotely at the cloud.

The deep neural network (DNN) $\mathcal{N}$ comprises L layers indexed by $l \in \{1, \ldots, L\}$. Layer l takes an input feature map $\mathbf{F}_l \in \mathbb{R}^{H'_l \times W'_l \times C'_l}$ and produces an output feature map $\mathbf{O}_l \in \mathbb{R}^{H_l \times W_l \times C_l}$; by convention $\mathbf{F}_1 = I_t$. The output at spatial position (i, j) depends on a set of input positions called its receptive field, denoted $\mathcal{R}^l(i, j) \subseteq \{1, \ldots, H'_l\} \times \{1, \ldots, W'_l\}$, with radius r_l. For convolutional and linear layers, $\mathbf{w}^l$ denotes the weight tensor applied over this receptive field, and s^l denotes the spatial stride. Because our analysis concerns only spatial positions, we omit the channel dimension hereafter. A contiguous group of feature-map positions (e.g., 16×16 pixels at the input) that correspond to a single MV entry is called a *shard*.

Let A_t denote the task accuracy (e.g., mAP) achieved on frame I_t under the realized reuse and endpoint selection, and let A^*_t denote the accuracy of full recomputation without caching. Let T_t denote the corresponding per-frame end-to-end latency, comprising local computation, data transmission, and remote computation as applicable. We define the sequence-level averages $\bar{A} = \frac{1}{T}\sum_{t=1}^{T} A_t$, $\bar{A}^* = \frac{1}{T}\sum_{t=1}^{T} A^*_t$, and $\bar{T} = \frac{1}{T}\sum_{t=1}^{T} T_t$. Because caching-based acceleration trades accuracy for speed, the following design goal guides all FluxShard mechanisms:

$$\min_{\{d_t\}} \bar{T} \quad \text{s.t.} \quad \bar{A} \geq \alpha \bar{A}^*, \tag{1}$$

where $d_t \in \{edge, cloud\}$ is the per-frame dispatch decision and $\alpha \in (0, 1]$ is a user-specified accuracy retention ratio.

At layer l, $\hat{\mathbf{F}}_l$ and $\hat{\mathbf{O}}_l$ denote the cached input and output features; since $\hat{\mathbf{O}}_l \equiv \hat{\mathbf{F}}_{l+1}$, we use whichever is more natural in context. At deeper layers, FluxShard downsamples the MV field from $\hat{\mathbf{m}}_0$ to match each layer's spatial resolution.

At each layer, FluxShard forms an *assembled output* $\tilde{\mathbf{O}}_l$ by combining MV-aligned cached values with freshly computed ones. Positions whose output discrepancy exceeds a per-layer tolerance τ_l require fresh computation and form the *recomputation set* $\mathcal{S}_l$. The thresholds $\boldsymbol{\tau} = \{\tau_l\}_{l=0}^{L}$ are calibrated offline,

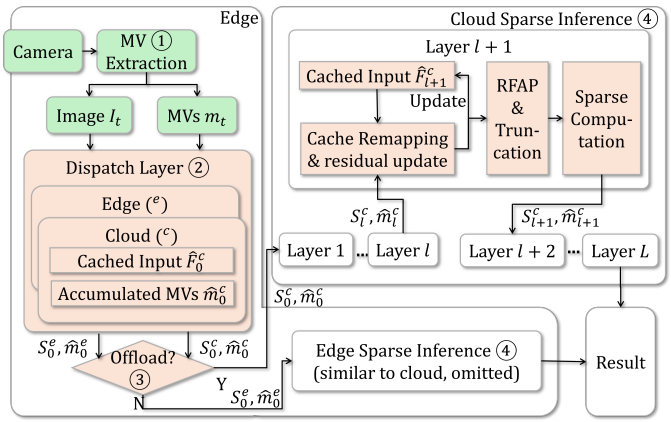


Figure 2. Overview of the FluxShard pipeline.

where $l = 0$ denotes a virtual layer that FluxShard prepends to the DNN for estimating the per-endpoint recomputation workload; the exact reuse criterion and calibration procedure are developed in the next section.

Because both endpoints execute the same inference logic, we present the reuse criterion and cache update rules without endpoint superscripts; $\mathcal{S}_l$, $\hat{\mathbf{F}}_l$, $\hat{\mathbf{O}}_l$ refer to the selected endpoint's state. Superscripts e or c appear only when the dispatch scheduler compares both endpoints.

B. Pipeline

Figure 2 illustrates the pipeline. FluxShard introduces a dispatch layer (virtual layer 0) preceding the DNN. This layer applies the identity operator to the raw input, so its reuse criterion reduces to a direct per-position comparison between the current frame and the MV-aligned cached input. For both endpoints, it stores a cached input ($\hat{\mathbf{F}}_0^e$, $\hat{\mathbf{F}}_0^c$) and an accumulated MV field ($\hat{\mathbf{m}}_0^e$, $\hat{\mathbf{m}}_0^c$) that tracks the total displacement from the cached input to the present frame. When an encoded frame arrives at the edge, FluxShard proceeds in the following four stages.

Stage 1: MV extraction. Block-level motion vectors $\mathbf{m}_t$ are extracted from the codec during decoding at negligible cost. Both dispatch states update their accumulated MV fields by warping the old accumulator along the new MV and adding the new displacement. When an endpoint performs inference, its accumulated field is reset; the other endpoint's field continues to accumulate. At deeper DNN layers, the MV field is downsampled from $\hat{\mathbf{m}}_0$ to match each layer's spatial resolution; we write $\hat{\mathbf{m}}_l$ for the field on the grid of layer l's output (equivalently layer $l+1$'s input).

Stage 2: Recomputation workload estimation. Each dispatch state uses its accumulated MV field to shift the cached input shard by shard, aligning each shard to its current-frame position. A per-position comparison within each aligned shard identifies positions whose content differs from the cache by more than the dispatch threshold τ_0; these positions form the candidate input recomputation sets $\mathcal{S}_0^e$ and $\mathcal{S}_0^c$. From their sizes, the edge state estimates local sparse inference latency, while the cloud state estimates the cost of transmitting the MV field and recomputation pixels plus remote inference under current network conditions.

Stage 3: Dispatch decision. The frame is assigned to the endpoint with the lower estimated latency; the accuracy constraint in Eq. (1) is enforced by the profiled thresholds $\boldsymbol{\tau}$ rather than by the routing step itself.

Stage 4: Inference and cache update. After dispatch, only the selected endpoint continues sparse inference. The selected endpoint begins from the layer-0 recomputation set and propagates it through the DNN one layer at a time. At each layer, the system remaps the cached features to the current-frame coordinate system, identifies which positions can be safely reused and which must be recomputed, and sparsely evaluates only the latter. Freshly computed and cached values are assembled into a result that serves as both the next layer's input and the updated cache. After the final layer, the endpoint resets its accumulated MV field.

The non-selected endpoint retains its accumulated MV field for future reuse. When the cloud is selected, the edge transmits only $\hat{\mathbf{m}}_0^c$ and the content-change recomputation pixels in $\mathcal{S}_0^c$; the cloud applies the RFAP check from the MV field before starting DNN inference.

These four stages rely on three mechanisms developed in the next section: per-layer thresholds $\boldsymbol{\tau}$ that control the accuracy-latency tradeoff by determining the recomputation set size, RFAP that ensures reuse correctness under heterogeneous motion, and cache remapping that keeps the reference aligned across frames.

IV. System Design

This section describes how FluxShard realizes the design goal in Eq. (1). We first reduce it to a per-frame recomputation minimization problem based on cache reuse (§IV-A). We then formalize the per-position reuse criterion (§IV-B) and present RFAP, which makes MV-aligned reuse correct under heterogeneous motion (§IV-C). Next, we describe threshold calibration and cache remapping, which respectively shrink the recomputation workload and preserve a valid cache reference (§IV-D). Finally, we integrate these mechanisms into a per-frame pipeline with dispatch (§IV-E).

A. From Design Goal to Minimal Feasible Recomputation Set

The design goal (Eq. (1)) minimizes average latency $\bar{T}$ subject to $\bar{A} \geq \alpha \bar{A}^*$. In principle, the optimal solution couples all frames: each frame's reuse and dispatch decisions determine the cache state inherited by subsequent frames, so minimizing $\bar{T}$ globally requires knowledge of future motion and content. In an online streaming setting this knowledge is unavailable, so FluxShard adopts a per-frame greedy relaxation. Requiring $A_t \geq \alpha A_t^*$ at every frame is a sufficient condition for the sequence-level accuracy constraint, and minimizing T_t per frame yields:

$$\min_{\mathcal{S}^e, \mathcal{S}^c} \min\{T_t^e(\mathcal{S}^e),\ T_t^c(\mathcal{S}^c,\ \hat{B})\} \quad \text{s.t.} \quad A_t \geq \alpha A_t^*, \quad (2)$$

where $\mathcal{S}^e = \{\mathcal{S}_l^e\}_{l=0}^L$ and $\mathcal{S}^c = \{\mathcal{S}_l^c\}_{l=0}^L$ are the collections of per-layer recomputation sets on each endpoint, computed over

independent cache states, and $\hat{B}$ is the current uplink bandwidth estimate. The inner $\min$ is the dispatch decision: the frame is routed to the faster endpoint. The outer minimization requires each endpoint to minimize its own T_t. Since per-endpoint latency is monotonically non-decreasing in the recomputation set size—the edge path incurs sparse computation determined by $\{\mathcal{S}_l^e\}$, while the cloud path additionally incurs transmission of $\mathcal{S}_0^c$—minimizing T_t^e and T_t^c each reduces to minimizing the recomputation sets:

$$\min_{\mathcal{S}_0,\,\{\mathcal{S}_l\}} \; |\mathcal{S}_0| + \textstyle\sum_l |\mathcal{S}_l| \quad \text{s.t.} \quad A_t \;\geq\; \alpha\, A_t^*. \tag{3}$$

Because both endpoints execute the same reuse logic, Eq. (3) applies to each independently; the system constructs the smallest feasible recomputation workload on each endpoint and routes the frame to the cheaper one.

B. Reuse Criterion

The core of Eq. (3) is deciding, at each layer and each position, whether the cached value can substitute for fresh computation without violating the accuracy constraint. As shown in §II-B, MV alignment removes displacement-induced false changes and improves cache reuse, but naïve reuse of MV-aligned features still degrades accuracy. This subsection establishes a per-position reuse criterion that formalizes when reuse is safe and identifies the source of the accuracy loss.

As introduced in §III-A, three quantities coexist at every output position (i,j) of layer l: the ground-truth output $\mathbf{O}_l(i,j)$, obtained by applying layer l to the current input $\mathbf{F}_l$ (or, in practice, to the assembled input $\tilde{\mathbf{O}}_{l-1}$); the cached output $\hat{\mathbf{O}}_l(\hat{\imath},\hat{\jmath})$ from a previous frame, computed from $\mathcal{R}^l(\hat{\imath},\hat{\jmath})$ in the cached input; and the assembled output $\tilde{\mathbf{O}}_l(i,j)$ is consumed by layer $l{+}1$. Reuse at position (i,j) is *valid* when substituting the cached value introduces bounded error:

$$\big|\, \mathbf{O}_l(i,j) - \hat{\mathbf{O}}_l(\hat{\imath},\hat{\jmath}) \big| \;\leq\; \tau_l, \tag{4}$$

where setting $\tau_l = 0$ reduces the criterion to exact output equality. Based on this criterion, FluxShard assembles the output of layer l as

$$\tilde{\mathbf{O}}_l(i,j) = \begin{cases} \hat{\mathbf{O}}_l(\hat{\imath},\hat{\jmath}), & \text{if Eq. (4) holds,} \\ \mathbf{O}_l(i,j), & \text{otherwise.} \end{cases} \tag{5}$$

The assembled output serves as the input to layer $l{+}1$. Positions assigned to the "otherwise" branch form the recomputation set $\mathcal{S}_l$.

Eq. (4) cannot be checked directly without computing $\mathbf{O}_l(i,j)$, which defeats the purpose of reuse. Because the cached output at $(\hat{\imath},\hat{\jmath})$ was produced from $\mathcal{R}^l(\hat{\imath},\hat{\jmath})$ in the cached input, reuse is safe whenever the current input within $\mathcal{R}^l(i,j)$ matches that cached patch. For each $(p,q) \in \mathcal{R}^l(i,j)$, let $(\hat{p},\hat{q})$ denote the corresponding position in $\mathcal{R}^l(\hat{\imath},\hat{\jmath})$ (i.e., at the same kernel offset). Define

$$\Delta^l_{\max}(i,j) \;=\; \max_{(p,q)\,\in\,\mathcal{R}^l(i,j)} \big|\, \mathbf{F}_l(p,q) - \hat{\mathbf{F}}_l(\hat{p},\hat{q}) \big| \tag{6}$$

denote the maximum absolute input perturbation over all spatial positions in the receptive field at output position (i,j). For a linear layer with weight vector $\mathbf{w}^l$ (the kernel flattened over all N input elements in the receptive field), linearity yields

$$\big|\, \mathbf{O}_l(i,j) - \hat{\mathbf{O}}_l(\hat{\imath},\hat{\jmath}) \big| \;\leq\; \|\mathbf{w}^l\|_1 \cdot \Delta^l_{\max}(i,j), \tag{7}$$

so Eq. (4) is guaranteed whenever the input patch satisfies

$$\Delta^l_{\max}(i,j) \;\leq\; \frac{\tau_l}{\|\mathbf{w}^l\|_1}. \tag{8}$$

For element-wise activations with Lipschitz constant at most 1 (ReLU, sigmoid, etc.), the input-side bound directly implies the output-side one. Batch normalization layers are affine at inference time, so Eq. (7) applies directly. Each layer's τ_l bounds the single-layer error independently; the cumulative effect of cascaded approximations is controlled by the offline profiling that jointly calibrates all τ_l over complete forward passes discussed in following subsections.

Evaluating Eq. (8) over the full receptive field at every output position and every layer costs as much as executing the layer itself. Per-position reuse from the previous layer can reduce this cost: for $(p,q) \notin \mathcal{S}_{l-1}$, the assembled value equals the cached value at the position indicated by the input MV $\hat{\mathbf{m}}_{l-1}(p,q)$, contributing zero to $\Delta^l_{\max}$. We call *reuse propagation* the practice of treating a position as reusable whenever its entire receptive field falls outside $\mathcal{S}_{l-1}$, which confines the evaluation of Eq. (8) to the neighborhood of recomputation positions. For layers with receptive field size one (pointwise convolutions, element-wise activations), this per-position check is sufficient: each output depends on exactly one input, so reuse at a position in the previous layer directly carries over to the same position at the current layer. For layers with receptive field size greater than one (e.g., 3×3 convolutions), however, output reuse at (i,j) requires the current receptive field $\mathcal{R}^l(i,j)$ to match the cached receptive field $\mathcal{R}^l(\hat{\imath},\hat{\jmath})$, and zero per-position input difference can only imply zero receptive field level difference if the input MV is uniform within the receptive field and geometrically coherent with the output MV.

C. Receptive Field Alignment Principle

Under heterogeneous per-block MVs, the above MV conditions are frequently violated, preventing reuse from propagating across layers with receptive field size greater than one. As shown in §II-B, ignoring such violations causes significant accuracy loss. To restore safe reuse propagation, the system must identify positions that violate the MV conditions and force them into the recomputation set. RFAP provides two sufficient conditions on the input-level MV field $\hat{\mathbf{m}}_0$ that identify all such positions in a single pass.

The first condition, *intra-receptive-field uniformity*, requires all input positions within the receptive field to share a single displacement:

$$\hat{\mathbf{m}}_{l-1}(p,q) = \hat{\mathbf{m}}_{l-1}(p',q'), \quad \forall\,(p,q),\,(p',q') \in \mathcal{R}^l(i,j). \tag{9}$$

A uniform shift ensures that the per-position cached values form the same contiguous patch $\mathcal{R}^l(\hat{\imath},\hat{\jmath})$ that produced the cached output.

The second condition, *input-output geometric coherence*, requires $\hat{\mathbf{m}}_l = \hat{\mathbf{m}}_{l-1}/s^l$ (assuming uniform stride in both dimensions). This ensures that the output MV and the (uniform) input MV point to the same cached content. When the displacement is not evenly divisible by the stride, the two fields point to different cached positions and the output must be recomputed.

When both conditions hold at a given output position, per-position reuse from the previous layer directly implies receptive field level correctness, and no further check is needed. Positions that violate either condition are forced into the recomputation set for recomputation, preserving accuracy.

a) Compaction to the input level.: Both conditions can be evaluated from the input-level MV field $\hat{\mathbf{m}}_0$ alone, because deeper-layer fields are obtained by downsampling $\hat{\mathbf{m}}_0$. For Condition 1, checking MV uniformity within the largest effective receptive field $R_{\max}$ (measured at the input grid) covers all layers: uniformity within $R_{\max}$ positions implies uniformity within any smaller receptive field at any intermediate layer. For Condition 2, checking that the displacement is divisible by the largest cumulative stride $S_{\max} = \max_l \prod_{k=1}^{l} s^k$ covers all individual strides. At the shard level, Condition 1 violations arise near shard boundaries where the $R_{\max}$-neighborhood spans multiple shards with different displacements; Condition 2 violations occur at shards whose displacement is not divisible by $S_{\max}$.

Input-level positions that fail either condition are merged into the recomputation set at the first DNN layer whose receptive field exceeds one. Because the dispatch layer is an identity operator with receptive field size one, these positions do not affect $\mathcal{S}_0$. At downstream layers, no separate MV check is needed: flagged positions are recomputed, and their fresh values propagate through the network via Eq. (8).

With $\tau_l = 0$ at all layers, the compacted check is provably correct: every structurally inconsistent position is recomputed. With profiled $\tau_l > 0$, some deeper-layer positions affected by resolved inconsistencies may exhibit small activation differences that truncation absorbs, avoiding unnecessary recomputation while staying within the calibrated accuracy budget.

D. Minimizing the Residual Set

The reuse criterion and RFAP establish how to efficiently determine reusable positions given a per-layer output discrepancy bound τ_l under heterogeneous motion. However, the relationship between τ_l and task accuracy A_t has not yet been established: larger thresholds yield smaller recomputation sets but risk violating the accuracy constraint in Eq. (3). This subsection addresses this gap: threshold calibration finds the largest feasible τ_l that keeps accuracy within budget, thereby minimizing the recomputation set; cache remapping then keeps the cache aligned so that subsequent frames also start from a valid reference.

1) Profiling-Driven Truncation: Even with RFAP and MV alignment, exact reuse ($\tau_l = 0$) leaves many low-magnitude discrepancies with negligible task impact. FluxShard assigns a nonzero τ_l to each profiled layer, truncating these discrepancies and reusing the cached value instead. A recomputation position is truncated whenever it satisfies the same input-side bound as Eq. (8) with the calibrated τ_l. In practice, nonzero thresholds are profiled only for the dispatch layer and a subset of DNN layers $\mathcal{L}_{\text{tr}}$ (selected activation layers); all other layers use $\tau_l = 0$. At layers with $\tau_l = 0$, only positions whose MV-aligned input has actually changed enter the recomputation set; the threshold merely forgoes the additional truncation of small changes, so sparsity remains high whenever the MV alignment successfully recovers displaced content. At the dispatch layer, the same rule uses the identity operator, i.e., $\|\mathbf{w}\|_1 = 1$, to form the input recomputation set $\mathcal{S}_0$.

The thresholds are jointly calibrated offline on the training set of each task. Let $\mathcal{L}_{\text{tr}} \subseteq \{1, \ldots, L\}$ denote the set of profiled DNN layers and let $K = |\mathcal{L}_{\text{tr}}|$. The total admissible accuracy drop relative to the dense baseline is

$$\Delta A = (1 - \alpha)\bar{A}^*. \tag{10}$$

Because the dispatch layer determines the input recomputation set and therefore all downstream workload—including the recomputation pixels transmitted when inference is offloaded—it has the greatest impact on latency and receives the larger share. The 2:1 ratio is empirically tuned on the training set. Concretely, we reserve two thirds of the budget for τ_0 and split the remaining one third evenly across the profiled DNN layers:

$$b_0 = \frac{2}{3}\Delta A, \qquad b_l = \frac{1}{3K}\Delta A, \;\; l \in \mathcal{L}_{\text{tr}}. \tag{11}$$

The dispatch layer is processed first, followed by the profiled DNN layers in network order. For each profiled stage $u \in \{0\} \cup \mathcal{L}_{\text{tr}}$, we search a discrete candidate set $\mathcal{C}^u$ and choose the largest threshold whose cumulative accuracy drop remains within the cumulative budget assigned up to that stage. This greedy search returns one calibrated τ_0 for input recomputation extraction and one τ_l for each profiled DNN layer. At runtime, the dispatch layer applies τ_0 to form $\mathcal{S}_0$, and each profiled DNN layer applies its calibrated τ_l using the same criterion.

2) Motion-Aware Cache Remapping: Threshold calibration minimizes the current frame's recomputation set, but subsequent frames depend on how well the cache reflects the current frame. Without realignment after each inference, the cache becomes stale: reusable positions no longer align with valid history, inflating the recomputation set, and positions declared reusable may consume corrupted cached features. As shown in §II-C, this drift causes the recomputation set to inflate over time, eventually exceeding even the cost of dense recomputation.

In video codecs, a reference frame is reconstructed by warping the previous reference along backward MVs and then merging newly decoded residuals; each position maps to exactly one source, so the warp is conflict-free. FluxShard applies this principle at the feature level: each current-frame shard looks up its source region in the cached features via backward MVs, then merges freshly computed values. At layer $l+1$, the cache update proceeds in two steps. First, the cached input $\hat{\mathbf{F}}_{l+1}$ is warped to the current-frame coordinate system using the accumulated MV field:

$$\hat{\mathbf{F}}_{l+1,\text{remap}}(i,j) \;=\; \hat{\mathbf{F}}_{l+1}\big((i,j) - \hat{\mathbf{m}}_l(i,j)\big). \tag{12}$$

Because the backward MV assigns each position to exactly one source, the warp is conflict-free with no unwritten holes. Second, the remapped cache is merged with the freshly computed outputs from layer l:

$$\hat{\mathbf{F}}_{l+1}(i,j) \;\leftarrow\; \begin{cases} \mathbf{O}_l(i,j), & (i,j) \in \mathcal{S}_l, \\ \hat{\mathbf{F}}_{l+1,\mathrm{remap}}(i,j), & \text{otherwise.} \end{cases} \tag{13}$$

Residual positions receive freshly computed values; reusable positions retain the warped cache. The resulting tensor replaces $\hat{\mathbf{F}}_{l+1}$ (equivalently $\hat{\mathbf{O}}_l$) as the new cached state.

After the update, the cache resides in the current-frame coordinate system, and the endpoint resets its accumulated MV field. Subsequent lookups start from identity alignment, avoiding double-shifting the remapped cache. The same remap-then-update rule applies to the input cache and to all intermediate layer caches maintained by the sparse runtime.

E. Per-Frame Pipeline and Dispatch

The preceding subsections solve Eq. (3) on each endpoint: the reuse criterion and RFAP enforce correctness, threshold calibration minimizes the recomputation sets under the accuracy constraint, and cache remapping maintains validity across frames. The remaining step in Eq. (2) is the dispatch: selecting the endpoint with lower T_t. Algorithm 1 integrates all preceding mechanisms into a per-frame pipeline. Lines 1–3 extract and accumulate the MV field into both endpoint states. Lines 4–7 compute the content-change recomputation sets and estimate per-endpoint latency. Lines 8–16 dispatch to the cheaper endpoint, apply the RFAP check (line 9/14), run sparse inference with profiled thresholds $\boldsymbol{\tau}$ (line 10/15), and update the cache (line 11/16). We now detail the dispatch decision.

For each incoming frame, both dispatch-layer states (§III-B) incorporate the new MV field $\mathbf{m}_t$ into their accumulated fields:

$$\hat{\mathbf{m}}_0(i,j) \;\leftarrow\; \hat{\mathbf{m}}_0\big((i,j) - \mathbf{m}_t(i,j)\big) \;+\; \mathbf{m}_t(i,j), \tag{14}$$

where the composition warps the old accumulator to the current coordinate system before adding the new displacement. Each state then extracts its recomputation set by applying the reuse criterion with the dispatch-layer threshold τ_0 and the identity operator ($\|\mathbf{w}\|_1 = 1$):

$$\mathcal{S}_0 \;=\; \big\{(i,j) : |\, I_t(i,j) - \hat{\mathbf{F}}_0\big((i,j) - \hat{\mathbf{m}}_0(i,j)\big)| > \tau_0\big\}. \tag{15}$$

Let $\rho_e = |\mathcal{S}_0^e|/N_{\mathrm{px}}$ and $\rho_c = |\mathcal{S}_0^c|/N_{\mathrm{px}}$ denote the normalized recomputation set sizes. The bandwidth estimate $\hat{B}$ is maintained as an exponentially weighted moving average of recent uplink measurements. FluxShard profiles the latency-vs-sparsity relationship offline on each endpoint, yielding:

$$T_{\mathrm{edge}} \;=\; f_{\mathrm{edge}}(\rho_e), \tag{16}$$

$$T_{\mathrm{cloud}} \;=\; f_{\mathrm{cloud}}(\rho_c) \;+\; |\mathcal{S}_0^c| \,/\, \hat{B}, \tag{17}$$

where f_{edge} and f_{cloud} are the profiled curves. The scheduler selects the endpoint with lower total latency; when the two estimates are within a margin ϵ, it prefers cloud offload to reduce edge energy.

Algorithm 1: FluxShard Per-Frame Pipeline

Input: Current frame I_t; edge dispatch-layer state ($\hat{\mathbf{F}}_0^e$, $\hat{\mathbf{m}}_0^e$); cloud dispatch-layer state ($\hat{\mathbf{F}}_0^c$, $\hat{\mathbf{m}}_0^c$); profiled thresholds τ_0, $\boldsymbol{\tau}$; latency profiles f_{edge}, f_{cloud}

Output: Inference result $\mathbf{y}_t$

`// Stage 1: MV extraction`

1 Extract MV field $\mathbf{m}_t$ from codec for frame I_t;

2 $\hat{\mathbf{m}}_0^e \leftarrow \texttt{Accumulate}(\hat{\mathbf{m}}_0^e, \mathbf{m}_t)$;

3 $\hat{\mathbf{m}}_0^c \leftarrow \texttt{Accumulate}(\hat{\mathbf{m}}_0^c, \mathbf{m}_t)$;

`// Stage 2: Recomputation workload estimation`

4 $\mathcal{S}_0^e \leftarrow \texttt{RecomputeSet}(I_t, \hat{\mathbf{F}}_0^e, \hat{\mathbf{m}}_0^e, \tau_0)$; `// Eq. (15)`

5 $\mathcal{S}_0^c \leftarrow \texttt{RecomputeSet}(I_t, \hat{\mathbf{F}}_0^c, \hat{\mathbf{m}}_0^c, \tau_0)$;

6 $\rho_e \leftarrow |\mathcal{S}_0^e|/|I_t|$; $\rho_c \leftarrow |\mathcal{S}_0^c|/|I_t|$;

7 $T_{\mathrm{edge}} \leftarrow f_{\mathrm{edge}}(\rho_e)$; $T_{\mathrm{cloud}} \leftarrow f_{\mathrm{cloud}}(\rho_c) + |\mathcal{S}_0^c| \,/\, \hat{B}$;

`// Stage 3--4: Dispatch, inference, cache update`

8 **if** $T_{\mathrm{edge}} < T_{\mathrm{cloud}} - \epsilon$ **then**

9 $\quad \mathcal{S}_{\mathrm{rfap}} \leftarrow \texttt{RFAP}(\hat{\mathbf{m}}_0^e)$; `// Eq. (9)`

10 $\quad \mathbf{y}_t \leftarrow \texttt{SparseInfer}_{\mathrm{edge}}(\mathcal{S}_0^e \cup \mathcal{S}_{\mathrm{rfap}}, \boldsymbol{\tau})$;

11 $\quad$ Update edge cache via Eqs. (12)–(13); reset $\hat{\mathbf{m}}_0^e$;

12 **else**

13 $\quad$ Transmit $\mathcal{S}_0^c$ and $\hat{\mathbf{m}}_0^c$ to cloud;

14 $\quad \mathcal{S}_{\mathrm{rfap}} \leftarrow \texttt{RFAP}(\hat{\mathbf{m}}_0^c)$;

15 $\quad \mathbf{y}_t \leftarrow \texttt{SparseInfer}_{\mathrm{cloud}}(\mathcal{S}_0^c \cup \mathcal{S}_{\mathrm{rfap}}, \boldsymbol{\tau})$;

16 $\quad$ Update cloud cache via Eqs. (12)–(13); reset $\hat{\mathbf{m}}_0^c$;

17 **return** $\mathbf{y}_t$;

After dispatch, only the selected endpoint runs inference. The RFAP check is applied on the selected endpoint's MV field to identify structurally inconsistent positions, which are merged with $\mathcal{S}_0$ before sparse inference begins. Because the dispatch layer has receptive field size one, RFAP-flagged positions are pixel-level correct after MV alignment and need not be transmitted; when inference is offloaded, only $\mathcal{S}_0^c$ and $\hat{\mathbf{m}}_0^c$ are sent to the cloud, and the cloud applies the RFAP check locally from the received MV field. After inference, the cache is updated via Eqs. (12)–(13) and the accumulated MV field is reset.

V. Implementation

We prototype FluxShard on an NVIDIA Jetson Xavier NX (edge) and an NVIDIA RTX 3080 GPU (cloud), connected over a TCP link whose bandwidth is shaped with `tc` to emulate cellular uplink conditions.

a) State management and offload reconstruction: The client maintains two dispatch-layer states as described in §III-B. When a frame is offloaded, the client sends the cloud state's accumulated MV field, a packed recomputation mask, and the recomputation RGB pixels. The server reconstructs the sparse input from these payloads and its cached reference, applies the RFAP check, runs sparse inference, and performs cache remapping. After transmission, the client mirrors the same cloud-state update to keep both sides synchronized.

b) RFAP consistency check: A custom CUDA kernel implements the RFAP check (Conditions 1 and 2 in §IV-C) directly from the accumulated MV field, producing a binary mask merged into the recomputation set at the first spatial layer.

c) Sparse inference engine: We store all feature tensors in channel-last (NHWC) layout throughout the sparse path, matching FluxShard's per-position access pattern and avoiding layout conversion overhead.

We implement sparse convolution, pooling, and activation with custom CUDA kernels. The convolution kernel follows an implicit-GEMM structure with the same dense-style load and store pattern as the dense counterpart, but skips arithmetic at inactive positions. At activation layers, truncation, MV-guided history lookup, and history refresh are fused into a single pass, so cache maintenance does not require a separate remapping kernel. On fully dense inputs, the custom convolution kernel approaches cuDNN-level throughput; as sparsity increases, execution time decreases nearly proportionally.

d) Offload metadata: FluxShard uses 16×16 MV blocks, so with two 16-bit components (int16) per block, the MV field occupies approximately 0.52% of the full RGB input in 8-bit format (uint8). For offloading, we further downsample the recomputation mask by 2×2 and pack it bitwise before transmission. The packed mask occupies approximately 1.04% of the input. Together, the MV field and mask contribute only 1.56% metadata overhead before the recomputation RGB pixels themselves are transmitted. These ratios are independent of the absolute input resolution.

VI. Evaluation

A. Experimental Setup

a) Testbed and network emulation: Figure 3 shows the testbed. The edge device is an NVIDIA Jetson Xavier NX with 384 CUDA cores and 8 GB LPDDR4x memory, embedded in a mobile robot. The cloud server hosts an NVIDIA RTX 3080 with 10 GB GDDR6X memory. Both machines run Ubuntu 20.04 with CUDA 11.4. They are connected through a 1 Gbps Ethernet switch, while the wireless uplink is emulated with Linux `tc`. We replay client-to-server bandwidth traces from a public 4G/5G measurement dataset [20] and group them into three tiers: *low*, *medium*, and *high*. The *low* tier uses 4G/LTE traces (40.4 ± 36.6 Mbps); the *medium* and *high* tiers are obtained by splitting the 5G traces at the median throughput into a lower half (382.8 ± 419.1 Mbps) and an upper half (596.9 ± 467.9 Mbps). The emulated link also adds a fixed 20 ms one-way propagation delay. Unless otherwise noted, we report results under the medium tier.

b) Workloads and datasets: We evaluate two real-time dense prediction workloads with the YOLO11 family [16]. We choose YOLO11 as a representative evaluation model because its backbone subsumes the spatial operation patterns of most convolutional architectures, including depthwise separable convolutions, residual blocks, etc. The two workloads are: *instance segmentation* (Seg) with YOLO11m-seg on DAVIS [25], a video object segmentation benchmark containing 90 sequences (25–104 frames each) depicting diverse objects in open-world scenes with varied camera motion, and *pose estimation* (Pose) with YOLO11m-pose on 3DPW [26], an outdoor human activity dataset captured from a handheld camera, containing 61 sequences (273–2178 frames each). All inputs are resized to 1024×1024. We use the official checkpoints without fine-tuning. Table I summarizes the two workloads; dense local inference on the edge device is already impractical for real-time use, while pure cloud inference is fast but dominated by network transport.

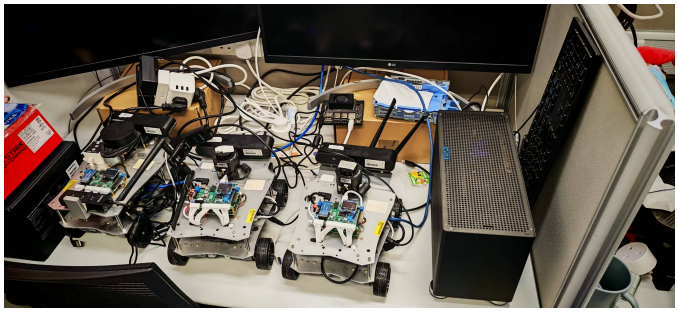


Figure 3. Testbed. Left: three mobile robots with embedded Jetson Xavier NX boards. Right: cloud server with RTX 3080.

Table I
Workload profile of the two evaluated tasks. Edge and server latency are dense (non-sparse) execution times. Edge energy is measured per frame via INA3221 power integration. MV std is the mean standard deviation of block-level motion vectors across frames, measuring motion intensity.

	Seg (YOLO11m-seg)	Pose (YOLO11m-pose)
Dataset	DAVIS	3DPW
Input resolution	1024×1024	1024×1024
Parameters (M)	22.4	20.9
Edge latency (ms)	537.5 ± 41.0	446.8 ± 16.0
Edge energy (J)	7.61	6.86
Server latency (ms)	35.7 ± 3.0	27.6 ± 3.0
MV std (px)	23.5	10.7

c) Motion vectors: DAVIS and 3DPW are distributed as image sequences rather than compressed video. To obtain realistic codec motion vectors, we encode each sequence with x264 in all-P-frame mode (a single leading I-frame followed by consecutive P-frames, no B-frames) and extract block-level MVs from the compressed bitstream at the standard 16×16 macroblock granularity. At runtime, the system consumes these codec-extracted motion vectors in the same way it would in a deployment where the camera feed is already H.264-encoded. As shown in Table I, DAVIS exhibits substantially stronger motion than 3DPW (MV std 23.5 vs. 10.7 px), reflecting its diverse camera motion and object dynamics; this makes it a more challenging setting for cache reuse. Together, the two datasets cover complementary temporal regimes: DAVIS's short sequences (25–104 frames) stress per-frame reuse under rapid motion, while 3DPW's long sequences (up to 2178 frames) test the long-term stability of cache remapping over extended operation.

d) Ground truth and metrics: For DAVIS segmentation, the native annotations are foreground-object masks for video object segmentation, which do not correspond to the YOLO

instance segmentation label set. We therefore generate pseudo ground truth by running dense YOLO11x-seg inference on every frame; all methods are compared against the same pseudo-GT, so the reported mIoU measures relative accuracy retention rather than absolute task accuracy, which is exactly what the constraint α in Eq. (1) requires. For 3DPW pose estimation, we use the dataset ground truth directly. We report mean Intersection over Union (mIoU) for segmentation, which measures the average overlap between predicted and ground-truth masks across all classes; higher is better. For pose estimation we report Object Keypoint Similarity (OKS), which measures the spatial agreement between predicted and ground-truth keypoints normalized by object scale; higher is better. End-to-end latency includes preprocessing, dispatch, transmission, inference, and cache maintenance. Edge energy is measured by sampling the board-level power rail at runtime and integrating over each frame interval. Because FluxShard may execute a frame either locally or through offload, the reported energy is the realized per-frame average over the mixed online execution path.

e) Baselines: We compare FluxShard against four baselines plus one local reference. **Local** executes dense inference entirely on the edge device; because it is bandwidth-insensitive, we use it only as a workload reference rather than a main baseline. **Offload** sends every full frame to the cloud server for dense inference, representing the pure-offload upper bound on network cost. **COACH** [8] is included as a representative of pipeline-level caching. It performs a whole-frame SSIM check for reuse decision: if the SSIM between the current frame and the cached reference exceeds 0.95, the entire output is reused; otherwise, the full frame is recomputed or transmitted, and the transmitted payload is quantized to one quarter of the original size. **DeltaCNN** [24] propagates only the per-pixel difference between consecutive frames through the network, skipping positions whose activation change is below a fixed threshold; it maintains the feature cache in a fixed coordinate system without motion compensation. **MotionDeltaCNN** [23] (referred to as M-DeltaCNN) extends DeltaCNN by estimating a single global homography from the frame pair and shifting the entire cache accordingly before computing deltas; it still treats the cache as a rigid whole, so heterogeneous local motion is not handled at the shard level. DeltaCNN uses the authors' official CUDA implementation, whereas MotionDeltaCNN is not open-sourced and is therefore re-implemented on top of the same sparse execution backend used by FluxShard. Consequently, comparisons between FluxShard and M-DeltaCNN isolate reuse semantics most directly, whereas absolute latency differences involving DeltaCNN also reflect engine differences; we revisit this point in the microbenchmark analysis. COACH, DeltaCNN, and M-DeltaCNN share the same online dispatch logic as FluxShard: the client estimates edge-local latency and cloud-offload latency (transmission plus server inference) from profiled sparsity-ratio lookup tables, then routes each frame to whichever endpoint is faster. This ensures the comparison isolates reuse semantics rather than transport or scheduling differences. All reported statistics exclude the first frame of each video sequence, which performs full initialization and does not benefit from caching.

B. End-to-End Performance

a) Latency and energy under dynamic bandwidth: Figure 4 compares the realized online latency and edge energy across bandwidth tiers. FluxShard achieves the lowest latency and energy across both workloads and all tiers. Motion-aligned cache remapping keeps the recomputation set small, which simultaneously reduces the transmitted payload and the computation on both endpoints; the dispatcher then routes the smaller workload to whichever endpoint is cheaper.

For segmentation, FluxShard delivers 39.9–80.4%, 36.8–69.0%, and 33.0–63.0% latency reduction under low, medium, and high bandwidth respectively, with 38.4–64.0%, 28.3–45.2%, and 23.7–51.0% energy savings. Under low bandwidth, segmentation latency drops from 955 ms (pure offload) to 187 ms, and energy from 4.43 J to 1.59 J. For pose estimation, the gains are larger: 46.4–83.8%, 43.0–74.4%, and 32.6–65.0% latency reduction, with 38.4–58.8%, 26.6–60.4%, and 14.9–59.2% energy savings. Pose latency drops from 832 ms to 134 ms and energy from 3.77 J to 1.55 J compared with Offload under low bandwidth. The advantage is widest under low bandwidth, where transmission dominates and FluxShard's small recomputation payload cuts the bottleneck directly.

Among the reuse baselines, the relative ranking between COACH and the DeltaCNN variants shifts with bandwidth. Under low bandwidth, COACH's quantization compresses the full-frame payload enough to beat DeltaCNN on segmentation (312 ms vs. 525 ms), because both DeltaCNN variants lose sparsity under strong motion and their recomputation payloads saturate the link. Under medium and high bandwidth, M-DeltaCNN's motion-compensated sparsity helps it overtake COACH on both workloads, while DeltaCNN sometimes still trails COACH on segmentation where camera motion is more diverse. FluxShard's advantage holds across all settings because its shard-level MV alignment preserves reuse under motion that invalidates fixed-coordinate or whole-frame approaches.

b) Accuracy: Table II reports accuracy across all settings. Our profiling-based configuration targets at most 3% accuracy loss relative to dense offload (the constraint $\bar{A} \geq \alpha \bar{A}^*$ in Eq. (1)). FluxShard stays within 2.0–2.7% across both workloads and all tiers, meeting the budget with margin. DeltaCNN and M-DeltaCNN lose 1.6–2.9%; DeltaCNN retains slightly higher accuracy on segmentation (1.6–1.7% vs. FluxShard's 2.0–2.7%) because its fixed-coordinate cache avoids the additional approximation introduced by MV alignment and truncation, at the cost of substantially higher latency. COACH incurs the largest drop (5.4–7.2%): even with a high SSIM threshold (0.95), whole-frame similarity does not ensure accurate dense predictions because small pixel-level changes can affect outputs such as segmentation masks and keypoint locations. Quantization of the transmitted frames further compounds the loss. We note that COACH was originally designed for recognition tasks where the output is a single class label per frame, making it more tolerant of output-level variation than the dense prediction tasks evaluated here.

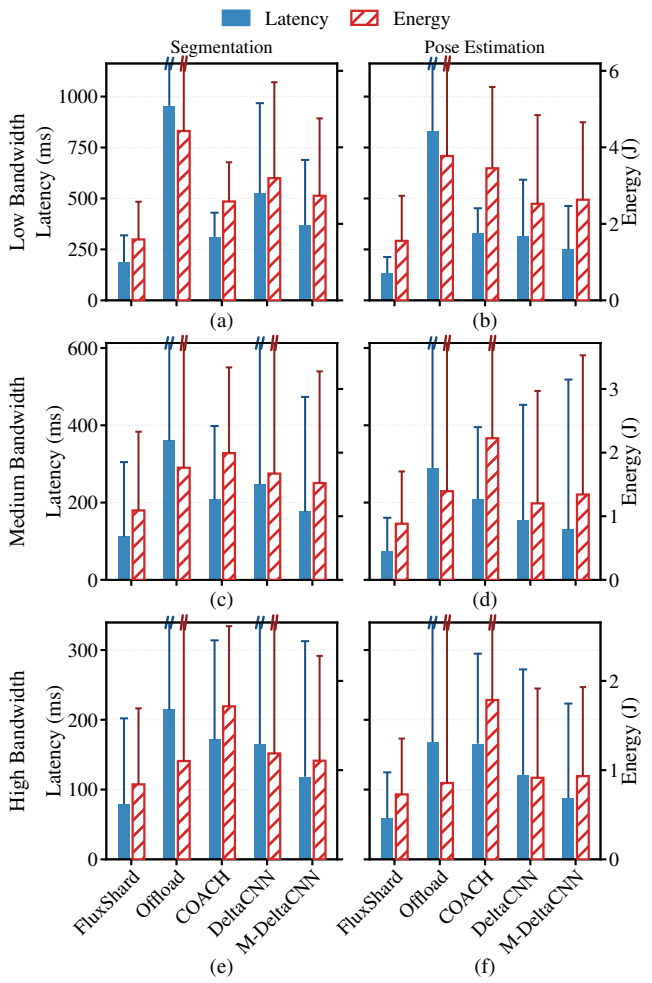


Figure 4. End-to-end latency (solid bars, left axis) and edge energy (hatched bars, right axis) under three bandwidth tiers. FluxShard achieves up to 83.8% latency reduction over pure offload and up to 64.5% over the best reuse baseline. Within each row, the two panels share the same axis range for direct comparison. Error bars show one standard deviation; break markers indicate clipped bars.

C. Microbenchmark

To understand where the end-to-end gains come from, we examine per-frame component statistics under medium bandwidth.

a) Ratio breakdown: Table III reports transmission ratio (fraction of full frame transmitted), computation ratio (fraction of dense FLOPs executed), and cloud dispatch ratio (fraction of frames routed to cloud) under medium bandwidth for both workloads. Across all methods, segmentation transmission and computation ratios are consistently higher than pose, reflecting the stronger motion in DAVIS (MV std 23.5 vs. 10.7 px, Table I): more motion invalidates more cached positions, increasing both the transmitted payload and the computation.

FluxShard transmits only 8.9–17.1% of the full frame and executes 40.7–47.9% of dense FLOPs, compared with 28.9–56.9% and 54.6–66.8% for the DeltaCNN variants. M-DeltaCNN consistently outperforms DeltaCNN on both transmission and computation ratios across both workloads, confirming that the global homography compensation in our re-implementation is functioning as intended. COACH transmits 24% of the full frame owing to its $4\times$ quantization, but still computes over 96% of dense FLOPs because its whole-frame SSIM check rarely triggers reuse on dynamic video. In contrast, FluxShard achieves a lower transmission ratio while also cutting computation by more than half, with only 2.0–2.7% accuracy loss (Table II). The cloud dispatch ratio is high for all methods (93–100%), reflecting the large compute gap between the Jetson Xavier NX (384 CUDA cores) and the RTX 3080 (8704 CUDA cores); under a weaker server or stronger edge device, the dispatcher would shift more frames to local execution. The last column of Table III reports peak GPU memory on the inference endpoint. FluxShard's feature caches add 136 MB over dense inference for segmentation and 72 MB for pose, comparable to DeltaCNN (146 MB and 112 MB). All methods stay below 2.3 GB, well within the 8 GB budget of the Jetson Xavier NX.

b) Latency breakdown: Figure 5 decomposes mean per-frame latency into preprocessing, transmission, and inference. The ratio differences in Table III translate directly into latency components. FluxShard has the smallest total bar on both workloads: its low transmission ratio and low computation ratio jointly shrink both the network and compute components. COACH's bar is dominated by transmission: although its payload is quantized to 24% of the full frame, it must recompute nearly all positions, so inference remains close to dense execution. COACH also exhibits a noticeable preprocessing cost because the whole-frame SSIM comparison is expensive on the resource-constrained Jetson Xavier NX. DeltaCNN and

Table II
END-TO-END ACCURACY UNDER TRACE-DRIVEN BANDWIDTH REPLAY. PARENTHESIZED VALUES SHOW THE PERCENTAGE DROP RELATIVE TO DENSE OFFLOAD.

Workload	Method	Low	Medium	High
Seg (mIoU)	Offload	69.15	69.15	69.15
	FluxShard	67.75 (-2.02%)	67.71 (-2.08%)	67.30 (-2.68%)
	COACH	65.42 (-5.40%)	65.18 (-5.74%)	64.87 (-6.19%)
	DeltaCNN	68.06 (-1.58%)	68.07 (-1.57%)	67.99 (-1.68%)
	M-DeltaCNN	67.25 (-2.75%)	67.19 (-2.84%)	67.17 (-2.86%)
Pose (OKS)	Offload	70.95	70.95	70.95
	FluxShard	69.21 (-2.45%)	69.13 (-2.56%)	69.02 (-2.72%)
	COACH	67.14 (-5.37%)	66.19 (-6.71%)	65.85 (-7.19%)
	DeltaCNN	69.00 (-2.75%)	69.02 (-2.72%)	69.00 (-2.75%)
	M-DeltaCNN	69.22 (-2.44%)	69.03 (-2.71%)	68.93 (-2.84%)

Table III
TRANSMISSION RATIO (FRACTION OF FULL FRAME TRANSMITTED), COMPUTATION RATIO (FRACTION OF DENSE FLOPS EXECUTED), AND CLOUD DISPATCH RATIO (FRACTION OF FRAMES ROUTED TO CLOUD) UNDER MEDIUM BANDWIDTH.

Method	Seg			Pose		
	Tx (%)	Comp (%)	Cloud (%)	Tx (%)	Comp (%)	Cloud (%)
FluxShard	17.1	47.9	98.1	8.9	40.7	95.6
Offload	100.0	100.0	100.0	100.0	100.0	100.0
COACH	24.0	96.0	97.5	24.1	96.5	93.4
DeltaCNN	56.9	66.8	98.4	31.7	54.6	96.4
M-DeltaCNN	45.5	65.7	98.0	28.9	58.5	93.7

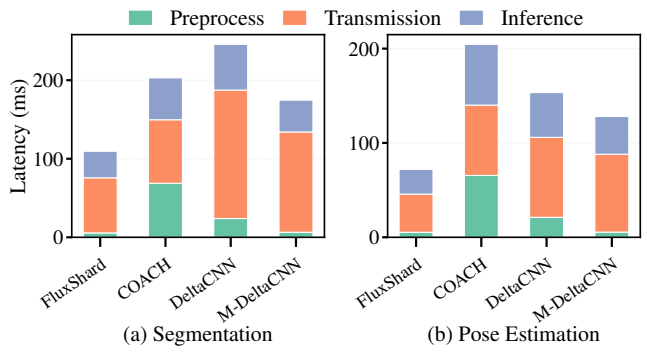


Figure 5. Mean per-frame latency breakdown under medium bandwidth. Preprocessing includes both edge-side and server-side preprocessing. DeltaCNN uses its official CUDA engine; all other sparse methods use our backend.

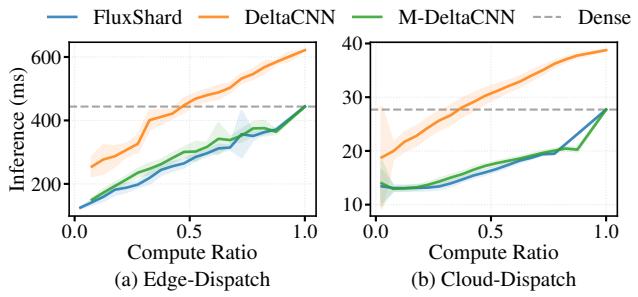


Figure 6. Inference latency vs. compute ratio for pose estimation under medium bandwidth, split by dispatch path. Dashed lines show dense inference latency on each endpoint. FluxShard and M-DeltaCNN share our sparse execution backend (overlapping trends); DeltaCNN uses its official CUDA implementation (different absolute level but similar slope).

M-DeltaCNN reduce both transmission and inference through delta-based sparsity, but their whole-cache semantics still leave a substantial active region under camera motion: DeltaCNN keeps a fixed-coordinate cache, while M-DeltaCNN applies only one global homography and therefore cannot follow locally heterogeneous motion. Despite similar computation ratios (Table III), DeltaCNN and M-DeltaCNN show different inference latencies; we explain this gap in the sparsity-to-latency analysis below.

c) Sparsity-to-latency relationship: Figure 6 plots inference latency against compute ratio for pose estimation under medium bandwidth, split by dispatch path. The horizontal dashed line marks the dense inference latency on each endpoint. FluxShard and M-DeltaCNN both use our sparse execution backend, so their trends overlap closely; the latency difference between them comes from the compute ratio each method achieves, not from engine-level overhead. DeltaCNN uses its own official CUDA implementation, which follows a similar near-linear trend but at a different absolute level. This explains the latency gap between DeltaCNN and M-DeltaCNN observed in Figure 5: although their computation ratios are similar, the underlying engines differ. The near-linear trend confirms that FluxShard's low compute ratio translates into proportionally low inference latency. This is particularly important under high bandwidth, where transmission is no longer the bottleneck and the end-to-end advantage increasingly depends on reducing computation time.

D. Ablation Study

Table IV ablates key FluxShard components under medium bandwidth. The default FluxShard uses the compacted input-level RFAP check; we compare against four variants.

a) RFAP variants: We compare three RFAP configurations: disabled, compacted input-level (default), and per-layer. Disabling RFAP entirely (w/o RFAP) yields the lowest compute ratio (Seg 45.0%, Pose 38.7%) because no positions are forcibly invalidated, but accuracy degrades substantially (Seg -3.29%, Pose -5.13%), with Pose exceeding the 3% budget. Without RFAP, positions where the input and output MV correspondences disagree are silently reused, and the resulting errors accumulate through the network. The impact is smaller on Seg because its stronger motion already produces a larger input-level active set (Seg tx ratio 17.1% vs. Pose 8.9%, Table III), which is more likely to overlap with the positions RFAP would have invalidated.

Per-layer RFAP checks MV consistency at every layer independently, achieving the highest accuracy among the three variants (Seg -1.75%, Pose -1.87%) but inflating the compute ratio to Seg 61.9% and Pose 48.8%, with latency also rising to Seg 117.1 ms and Pose 75.6 ms. The redundant per-layer checks force recomputation at deeper-layer positions whose actual discrepancy magnitude is small enough for truncation to handle.

The compacted input-level check (default) strikes the best trade-off: it captures all structurally inconsistent positions at the input level where MV heterogeneity is explicit, and lets the profiled thresholds $\{\tau_l\}$ absorb small discrepancies at deeper layers. This keeps the compute ratio at Seg 47.9% and Pose 40.7% with accuracy drop of Seg -2.08% and Pose -2.56%.

b) w/o offload: Restricting all inference to the edge device (w/o offload) yields Seg -0.30% and Pose -2.94% accuracy drop. Latency, however, increases dramatically: Seg rises to 433.9 ms and Pose to 252.4 ms, a 3.4–3.9× slowdown, confirming that sparse reuse alone cannot compensate for the loss of cloud compute capacity.

c) w/o sparse: Disabling sparse computation while keeping the original transmission logic (w/o sparse) yields Seg -1.90% and Pose -0.23% accuracy drop. Latency rises to Seg 124.9 ms and Pose 92.0 ms, a 1.1–1.2× increase over the full system, because the server executes dense inference on every offloaded frame instead of reusing cached features.

d) w/o remap: Removing MV-guided cache remapping (w/o remap) causes the feature cache to drift out of alignment with the current frame over time. As the accumulated displacement grows, RFAP and truncation detect an increasing number of inconsistent positions and force recomputation, driving compute ratio to Seg 77.1% and Pose 77.3%. The same drift degrades the sparsity estimate used by the dispatcher, increasing the transmitted payload per frame; together, latency climbs to Seg 226.4 ms and Pose 198.1 ms—roughly 2× the default and exceeding even the w/o sparse baseline.

Table IV
ABLATION STUDY UNDER MEDIUM BANDWIDTH. FLUXSHARD USES COMPACTED INPUT-LEVEL RFAP AS THE DEFAULT CONFIGURATION. ACCURACY DROP IS RELATIVE TO DENSE OFFLOAD.

Workload	Variant	Acc (%)	Comp (%)	Lat (ms)
Seg (mIoU)	FluxShard	67.71 (-2.08%)	47.9	111.7
	w/o RFAP	66.87 (-3.29%)	45.0	115.0
	Per-layer RFAP	67.94 (-1.75%)	61.9	117.1
	w/o offload	68.94 (-0.30%)	45.4	433.9
	w/o sparse	67.84 (-1.90%)	100.0	124.9
	w/o remap	68.41 (-1.07%)	77.1	226.4
Pose (OKS)	FluxShard	69.13 (-2.56%)	40.7	74.1
	w/o RFAP	67.31 (-5.13%)	38.7	77.6
	Per-layer RFAP	69.62 (-1.87%)	48.8	75.6
	w/o offload	68.87 (-2.94%)	41.1	252.4
	w/o sparse	70.79 (-0.23%)	100.0	92.0
	w/o remap	63.42 (-10.61%)	77.3	198.1

Accuracy shows an asymmetric effect: Pose degrades to -10.61% because keypoint localization is particularly sensitive to coordinate-system misalignment, while Seg shows only -1.07%, marginally better than the default (-2.08%). At a compute ratio of 77.1%, the system recomputes the vast majority of positions, leaving almost no room for reuse errors to manifest; the near-dense execution incidentally eliminates the small approximation errors that the default's aggressive reuse introduces. The marginal accuracy difference (1 pp) is well within the noise floor of the 3% budget and comes at the cost of $2\times$ latency, confirming that cache remapping is essential for efficiency.

Overall, the default FluxShard configuration (compacted RFAP + sparse inference + edge-cloud dispatch) achieves the lowest latency across all variants while keeping accuracy within the 3% budget. No single component can be removed without either exceeding the accuracy budget or substantially increasing latency.

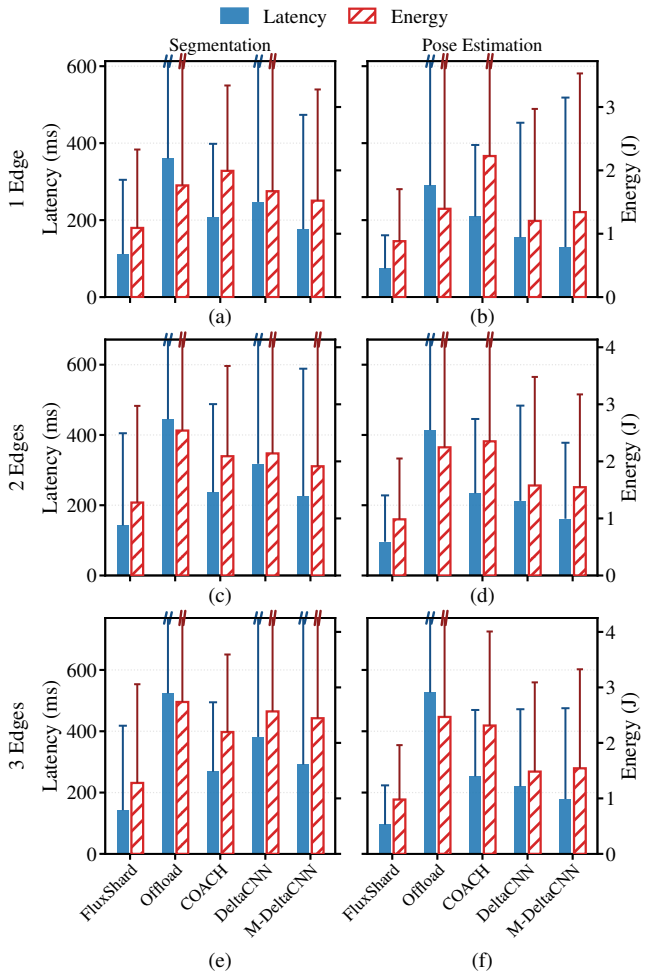


Figure 7. End-to-end latency (solid bars, left axis) and edge energy (hatched bars, right axis) under 1, 2, and 3 concurrent edge devices under medium bandwidth.

E. Scalability

To evaluate how FluxShard scales when multiple edge devices share the same cloud server, we run 1, 2, and 3 concurrent edges under medium bandwidth. Each additional edge runs as an independent client process on a separate Jetson Xavier NX; all clients share the same cloud server and the same `tc`-shaped uplink. Because each client maintains its own cache state and accuracy is determined by the per-client reuse policy (which is unchanged), we focus on latency and energy.

a) Latency and energy: Figure 7 compares latency and energy as the number of active edges increases. For pose estimation, FluxShard's latency rises from 74.1 ms (1 edge) to 96.0 ms (2 edges) and 95.2 ms (3 edges), a 28% increase that plateaus between 2 and 3 clients; the slight decrease from 2 to 3 edges is within run-to-run variance. Segmentation follows a similar pattern, rising from 111.7 ms to 142.4 ms at 2 edges. In contrast, pure Offload degrades from 289.8 ms to 526.6 ms (+82%) on pose and from 360.2 ms to 523.5 ms (+45%) on segmentation, as full-frame payloads from multiple clients contend for server compute and uplink bandwidth. Among the reuse baselines, M-DeltaCNN rises by 37% (pose) and DeltaCNN by 42% (pose), both larger than FluxShard's increase. FluxShard's advantage widens with more clients because its small recomputation payload (8.5–17.2% transmission ratio) imposes less load on the shared server than competing methods. Edge energy follows the same trend: FluxShard rises by only 11% on pose (882.5 mJ to 979.1 mJ), while Offload nearly doubles.

F. Discussion

When the camera is static, per-block MVs are near zero and cache remapping becomes a no-op; FluxShard degrades gracefully to behavior similar to DeltaCNN. Under extreme motion such as scene cuts, the reuse ratio approaches zero and FluxShard falls back to full recomputation; this is the expected lower bound, as no temporal reuse method can exploit non-existent temporal redundancy.

a) MV quality: Codec MVs are the output of block matching during encoding, not optical flow; in texture-flat regions or under occlusion, the encoder may select a displacement that does not reflect the true motion. Inaccurate MVs do not cause silent accuracy degradation, however, because FluxShard's correctness relies on two independent checks:

RFAP detects structural inconsistency within the receptive field, and the content-change threshold τ_0 detects value mismatches after MV alignment. If an MV points to the wrong source, the aligned pixel value will differ from the current frame, and the position will enter the recomputation set and be recomputed. The only consequence of inaccurate MVs is a lower reuse ratio, which increases latency but never compromises accuracy.

FluxShard's mechanisms are defined for convolutional architectures with fixed receptive fields and spatial strides. RFAP's two conditions depend only on receptive field geometry and layer strides, properties shared by all standard convolutional layers (regular, depthwise separable, dilated, and grouped convolutions) regardless of the surrounding network topology. Cache remapping relies solely on the MV field and backward warping, which operate on spatial grids independent of layer type. Because YOLO11's backbone subsumes the spatial operation patterns of most CNN-based detection and segmentation architectures [16], the evaluation spans sufficient structural diversity. Attention-based layers, however, aggregate information from data-dependent positions rather than a fixed spatial neighborhood, so Condition 1 (intra-receptive-field uniformity) does not directly apply. Extending RFAP to architectures with dynamic receptive fields is an open direction for future work.

VII. Related Work

Because consecutive video frames share substantial visual content, a growing body of work caches previously computed features and reuses them for subsequent frames, reducing both computation and transmission. Existing approaches fall into two broad categories.

Pipeline-level caching. Methods such as SPINN [17] and COACH [8] cache intermediate feature maps or predictions and reuse them when successive inputs are globally similar, making a *binary, whole-input* reuse decision. This coarse granularity suits classification but cannot approximate the spatially dense outputs required by segmentation or detection.

Delta-based sparse inference. A second line of work including RRM [22], CBinfer [2], Skip-Convolution [9], and DeltaCNN [24] maintains a pixel-level reference cache and propagates only the *difference* (delta) between the current frame and the reference through the network, computing only at affected output locations and reusing cached values elsewhere. Among these, DeltaCNN represents the most complete realization: it achieves end-to-end sparse propagation with truncation buffers that prevent error accumulation, enabling unbounded-length sequences without dense resets and pushing the efficiency of static-camera settings to its practical limit. MotionDeltaCNN [23] extends DeltaCNN to moving cameras by warping the cache with a single global homography before computing the delta. However, a global transformation cannot capture locally heterogeneous motion—independently moving objects, depth-induced parallax, or mixed ego-motion and object motion—leaving large residual deltas that erode sparsity.

Shared limitation. Across both categories, the cache is treated as an *indivisible, scene-level entity*: it is either globally valid, globally stale, or aligned uniformly using a single transformation. Real-world mobile video, however, exhibits spatially non-uniform motion: a camera pan shifts the background uniformly while foreground objects move independently, and depth discontinuities create parallax that no single warp can reconcile. MotionDeltaCNN [23] reported that their homography alignment succeeds on only a small fraction of the DAVIS [25] sequences (14 out of 80 [23]) evaluated. The result is a *granularity mismatch*: the cache granularity is the whole scene, but motion granularity is per-region. This mismatch forces existing methods to either waste computation re-deriving content that has merely shifted, or sacrifice correctness by reusing misaligned features.

Note that several other techniques also reduce the cost of video analytics, including ROI filtering [3], [18], input resolution adaptation [14], [5], frame sampling that skips inference on selected frames [3], [18], and model compression via quantization or pruning [12], [11]. Split-point inference methods such as Neurosurgeon [15] partition a DNN at an intermediate layer and transmit compressed features rather than raw pixels, reducing per-frame transmission size. These strategies are orthogonal to feature cache reuse: split-point methods reduce the cost of a *single* frame by compressing its intermediate representation, whereas caching exploits *temporal* redundancy across frames to avoid redundant computation and transmission altogether. FluxShard can be composed with any of them; this work focuses exclusively on the cache reuse axis.

VIII. Conclusion

We have presented FluxShard, a motion-aware edge-cloud video analytics system that uses codec-level motion vectors to manage feature cache reuse at the granularity of individual motion regions, addressing the mismatch between whole-scene cache management and local heterogeneous motion. Two mechanisms make this feasible: RFAP, which consolidates the per-layer MV consistency check into a single input-level pass by identifying where the per-position and receptive-field-level correspondences diverge; and motion-aware cache remapping, which uses backward-MV warping to keep the cache aligned across frames without write conflicts. On instance segmentation and pose estimation with YOLO11 across three bandwidth tiers, FluxShard achieves 32.6–83.8% latency reduction and 14.9–64.0% energy savings over all baselines while retaining 97.3–98.0% of dense-model accuracy. By enabling real-time, high-accuracy video analytics on resource-constrained mobile platforms, FluxShard can serve as an inference infrastructure for latency-sensitive edge applications such as autonomous drones, embodied robots, and augmented-reality devices.

References

[1] Hassan J. Al Dawasari, Muhammad Bilal, Muhammad Moinuddin, Kamran Arshad, and Khaled Assaleh. Deepvision: Enhanced drone detection and recognition in visible imagery through deep learning networks. *Sensors*, 23(21), 2023.

[2] Lukas Cavigelli, Philippe Degen, and Luca Benini. CBinfer: Change-based inference for convolutional neural networks on video data.

[3] Tiffany Yu-Han Chen, Lenin Ravindranath, Shuo Deng, Paramvir Bahl, and Hari Balakrishnan. Glimpse: Continuous, real-time object recognition on mobile devices. In *Proceedings of the 13th ACM Conference on Embedded Networked Sensor Systems*, SenSys '15, pages 155–168. Association for Computing Machinery.

[4] Hyomin Choi and Ivan V. Bajić. Scalable image coding for humans and machines. 31:2739–2754.

[5] Kuntai Du, Qizheng Zhang, Anton Arapin, Haodong Wang, Zhengxu Xia, and Junchen Jiang. AccMPEG: Optimizing video encoding for video analytics.

[6] Lingyu Duan, Jiaying Liu, Wenhan Yang, Tiejun Huang, and Wen Gao. Video coding for machines: A paradigm of collaborative compression and intelligent analytics. 29:8680–8695.

[7] Leonardo Galteri, Lorenzo Seidenari, Marco Bertini, and Alberto Del Bimbo. Deep generative adversarial compression artifact removal. In *2017 IEEE International Conference on Computer Vision (ICCV)*, pages 4836–4845. ISSN: 2380-7504.

[8] Luyao Gao, Jianchun Liu, Hongli Xu, Sun Xu, Qianpiao Ma, and Liusheng Huang. Accelerating end-cloud collaborative inference via near bubble-free pipeline optimization. *arXiv preprint arXiv:2501.12388*, 2024.

[9] Amirhossein Habibian, Davide Abati, Taco S. Cohen, and Babak Ehteshami Bejnordi. Skip-convolutions for efficient video processing. In *2021 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 2694–2703. ISSN: 2575-7075.

[10] Beining Han, Meenal Parakh, Derek Geng, Jack A. Defay, Gan Luyang, and Jia Deng. FetchBench: A simulation benchmark for robot fetching.

[11] Song Han, Jeff Pool, John Tran, and William J. Dally. Learning both weights and connections for efficient neural networks. In *Proceedings of the 29th International Conference on Neural Information Processing Systems - Volume 1*, volume 1 of *NIPS'15*, pages 1135–1143. MIT Press.

[12] Benoit Jacob, Skirmantas Kligys, Bo Chen, Menglong Zhu, Matthew Tang, Andrew Howard, Hartwig Adam, and Dmitry Kalenichenko. Quantization and training of neural networks for efficient integer-arithmetic-only inference. In *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 2704–2713. IEEE.

[13] Fatemeh Jalali, Morteza Khademi, Abbas Ebrahimi Moghadam, and Hadi Sadoghi Yazdi. Robust scene aware multi-object tracking for surveillance videos. 638:130114.

[14] Junchen Jiang, Ganesh Ananthanarayanan, Peter Bodik, Siddhartha Sen, and Ion Stoica. Chameleon: scalable adaptation of video analytics. In *Proceedings of the 2018 Conference of the ACM Special Interest Group on Data Communication*, SIGCOMM '18, pages 253–266. Association for Computing Machinery.

[15] Yiping Kang, Johann Hauswald, Cao Gao, Austin Rovinski, Trevor Mudge, Jason Mars, and Lingjia Tang. Neurosurgeon: Collaborative intelligence between the cloud and mobile edge. In *Proceedings of the Twenty-Second International Conference on Architectural Support for Programming Languages and Operating Systems*, ASPLOS '17, pages 615–629. Association for Computing Machinery.

[16] Rahima Khanam and Muhammad Hussain. Yolov11: An overview of the key architectural enhancements. *arXiv preprint arXiv:2410.17725*, 2024.

[17] Stefanos Laskaridis, Stylianos I Venieris, Mario Almeida, Ilias Leontiadis, and Nicholas D Lane. Spinn: synergistic progressive inference of neural networks over device and cloud. In *Proceedings of the 26th annual international conference on mobile computing and networking*, pages 1–15, 2020.

[18] Yuanqi Li, Arthi Padmanabhan, Pengzhan Zhao, Yufei Wang, Guoqing Harry Xu, and Ravi Netravali. Reducto: On-camera filtering for resource-efficient real-time video analytics. In *Proceedings of the Annual conference of the ACM Special Interest Group on Data Communication on the applications, technologies, architectures, and protocols for computer communication*, SIGCOMM '20, pages 359–376. Association for Computing Machinery.

[19] Mostafa Mahmoud, Kevin Siu, and Andreas Moshovos. Diffy: a déjà vu-free differential deep neural network accelerator. In *Proceedings of the 51st Annual IEEE/ACM International Symposium on Microarchitecture*, MICRO-51, pages 134–147. IEEE Press.

[20] Arvind Narayanan, Xumiao Zhang, Ruiyang Zhu, Ahmad Hassan, Shuowei Jin, Xiao Zhu, Xiaoxuan Zhang, Denis Rybkin, Zhengxuan Yang, Zhuoqing Morley Mao, Feng Qian, and Zhi-Li Zhang. A variegated look at 5g in the wild: performance, power, and QoE implications. In *Proceedings of the 2021 ACM SIGCOMM 2021 Conference*, SIGCOMM '21, pages 610–625. Association for Computing Machinery.

[21] Kien Nguyen, Feng Liu, Clinton Fookes, Sridha Sridharan, Xiaoming Liu, and Arun Ross. Person recognition in aerial surveillance: A decade survey. pages 1–1.

[22] Bowen Pan, Wuwei Lin, Xiaolin Fang, Chaoqin Huang, Bolei Zhou, and Cewu Lu. Recurrent residual module for fast inference in videos. In *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 1536–1545. ISSN: 2575-7075.

[23] Mathias Parger, Chengcheng Tang, Thomas Neff, Christopher D. Twigg, Cem Keskin, Robert Wang, and Markus Steinberger. MotionDeltaCNN: Sparse CNN inference of frame differences in moving camera videos with spherical buffers and padded convolutions. pages 17292–17301.

[24] Mathias Parger, Chengcheng Tang, Christopher D Twigg, Cem Keskin, Robert Wang, and Markus Steinberger. Deltacnn: End-to-end cnn inference of sparse frame differences in videos. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 12497–12506, 2022.

[25] Jordi Pont-Tuset, Federico Perazzi, Sergi Caelles, Pablo Arbeláez, Alexander Sorkine-Hornung, and Luc Van Gool. The 2017 davis challenge on video object segmentation. *arXiv:1704.00675*, 2017.

[26] Timo von Marcard, Roberto Henschel, Michael Black, Bodo Rosenhahn, and Gerard Pons-Moll. Recovering accurate 3d human pose in the wild using imus and a moving camera. In *European Conference on Computer Vision (ECCV)*, sep 2018.

[27] Qi Wu, Zipeng Fu, Xuxin Cheng, Xiaolong Wang, and Chelsea Finn. Helpful DoggyBot: Open-world object fetching using legged robots and vision-language models.

[28] Jun Zhang, Mina Henein, Robert Mahony, and Viorela Ila. VDO-SLAM: A visual dynamic object-aware SLAM system.